

P1135R0

The C++20 Synchronization Library

Published Proposal, 12 June 2018

This version:

<http://wg21.link/P1135r0>

Issue Tracking:

[GitHub](#)

Authors:

[Bryce Adelstein Lelbach](#) (NVIDIA)

[Olivier Giroux](#) (NVIDIA)

[JF Bastien](#) (Apple)

Audience:

SG1, LEWG, LWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Table of Contents

1 Introduction

2 Wording

Index

Terms defined by this specification

§ 1. Introduction

This paper is the unification of the wording for a series of related C++20 proposals for introducing new synchronization and thread coordination facilities and enhancing existing ones:

- [P0514]: Efficient `atomic` waiting and semaphores.
- [P0666]: Latches and barriers.
- [P0995]: `atomic_flag::test` and lockfree integral types.

§ 2. Wording

Note: The  character is used to denote a placeholder number which shall be selected by the editor.

Modify the header synopsis for `<atomic>` in [**atomics.syn**] as follows:

32.2 Header <atomic> synopsis

[atomics.syn]

```
namespace std {  
    // ...  
  
    // 32.3, type aliases  
    // ...  
  
    using atomic_intptr_t      = atomic<intptr_t>;  
    using atomic_uintptr_t     = atomic<uintptr_t>;  
    using atomic_size_t       = atomic<size_t>;  
    using atomic_ptrdiff_t    = atomic<ptrdiff_t>;  
    using atomic_intmax_t     = atomic<intmax_t>;  
    using atomic_uintmax_t    = atomic<uintmax_t>;  
  
    using atomic_int_fast_wait_t = atomic<implementation-defined>;  
    using atomic_uint_fast_wait_t = atomic<implementation-defined>;  
    using atomic_signed_lock_free   = see below;  
    using atomic_unsigned_lock_free = see below;  
  
    // ...  
  
    // 32.8, flag type and operations  
    struct atomic_flag;  
    bool atomic_flag_test(volatile atomic_flag*) noexcept;  
    bool atomic_flag_test(atomic_flag*) noexcept;  
    bool atomic_flag_test_explicit(volatile atomic_flag*, memory_order) noexcept;  
    bool atomic_flag_test_explicit(atomic_flag*, memory_order) noexcept;
```

```
bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
bool atomic_flag_test_and_set(atomic_flag*) noexcept;
bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_order) noexcept;
bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noexcept;
void atomic_flag_clear(volatile atomic_flag*) noexcept;
void atomic_flag_clear(atomic_flag*) noexcept;
void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noexcept;
void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;
#define ATOMIC_FLAG_INIT see below
```

```
// 32.9, fences
```

```
extern "C" void atomic_thread_fence(memory_order) noexcept;
extern "C" void atomic_signal_fence(memory_order) noexcept;
```

```
// 32.10, waiting and notifying functions
```

```

template<class T>
    void atomic_notify_one(const volatile atomic<T>*);
template<class T>
    void atomic_notify_one(const atomic<T>*);
void atomic_notify_one(const volatile atomic_flag*);
void atomic_notify_one(const atomic_flag*);
template<class T>
    void atomic_notify_all(const volatile atomic<T>*);
template<class T>
    void atomic_notify_all(const atomic<T>*);
void atomic_notify_all(const volatile atomic_flag*);
void atomic_notify_all(const atomic_flag*);
template<class T>
    void atomic_wait(const volatile atomic<T>*,
                     typename atomic<T>::value_type);
template<class T>
    void atomic_wait(const atomic<T>*,
                     typename atomic<T>::value_type);
void atomic_wait(const volatile atomic_flag*,
                 bool);
void atomic_wait(const atomic_flag*,
                 bool);
template<class T>
    void atomic_wait_explicit(const volatile atomic<T>*,
                              typename atomic<T>::value_type,
                              memory_order);
template<class T>
    void atomic_wait_explicit(const atomic<T>*,
                              typename atomic<T>::value_type,
                              memory_order);
void atomic_wait_explicit(const volatile atomic_flag*,
                          bool,
                          memory_order);
void atomic_wait_explicit(const atomic_flag*,
                          bool,
                          memory_order);
}

```

Modify [**atomics.alias**] as follows:

32.3 Type aliases

[atomics.alias]

- 1 The type aliases `atomic_intN_t`, `atomic_uintN_t`, `atomic_intptr_t`, and `atomic_uintptr_t` are defined if and only if `intN_t`, `uintN_t`, `intptr_t`, and `uintptr_t` are defined, respectively.
- 2 The type aliases `atomic_signed_lock_free` and `atomic_unsigned_lock_free` are defined to be specializations of `atomic` whose template arguments are integral types, respectively signed and unsigned, other than `bool`. `is_always_lock_free` shall be true for `atomic_signed_lock_free` and `atomic_unsigned_lock_free`. An implementation should choose the integral specialization of `atomic` for which the waiting and notifying functions are most efficient.
- 3 The type aliases `atomic_int_fast_wait_t` and `atomic_uint_fast_wait_t` are integral atomic types. Implementations should ensure that invocations of waiting and notifying functions (32.❖) with these types have the lowest performance overhead among integer types.

Note: The reference to "waiting and notifying functions" in the above change should refer to the new [atomic.wait] subclause.

Modify [atomics.flag] as follows:

32.8 Flag type and operations

[atomics.flag]

```
namespace std {
    struct atomic_flag {
        bool test(memory_order = memory_order_seq_cst) volatile noexcept;
        bool test(memory_order = memory_order_seq_cst) noexcept;
        bool test_and_set(memory_order = memory_order_seq_cst) volatile noexcept;
        bool test_and_set(memory_order = memory_order_seq_cst) noexcept;
        void clear(memory_order = memory_order_seq_cst) volatile noexcept;
        void clear(memory_order = memory_order_seq_cst) noexcept;

        atomic_flag() noexcept = default;
        atomic_flag(const atomic_flag&) = delete;
        atomic_flag& operator=(const atomic_flag&) = delete;
        atomic_flag& operator=(const atomic_flag&) volatile = delete;
    };

    bool atomic_flag_test(volatile atomic_flag*) noexcept;
    bool atomic_flag_test(atomic_flag*) noexcept;
    bool atomic_flag_test_explicit(volatile atomic_flag*, memory_order) noexcept;
    bool atomic_flag_test_explicit(atomic_flag*, memory_order) noexcept;

    bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
    bool atomic_flag_test_and_set(atomic_flag*) noexcept;
    bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_order) noexcept;
    bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noexcept;
    void atomic_flag_clear(volatile atomic_flag*) noexcept;
    void atomic_flag_clear(atomic_flag*) noexcept;
    void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noexcept;
    void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;

    #define ATOMIC_FLAG_INIT see below
}
```

- 1 The `atomic_flag` type provides the classic test-and-set functionality. It has two states, set and clear.
- 2 Operations on an object of type `atomic_flag` shall be lock-free. [*Note:* Hence the operations should also be address-free. — *end note*]
- 3 The `atomic_flag` type is a standard-layout struct. It has a trivial default constructor and a trivial destructor.

- 4 The macro `ATOMIC_FLAG_INIT` shall be defined in such a way that it can be used to initialize an object of type `atomic_flag` to the clear state. The macro can be used in the form:

```
atomic_flag guard = ATOMIC_FLAG_INIT;
```

It is unspecified whether the macro can be used in other initialization contexts. For a complete static-duration object, that initialization shall be static. Unless initialized with `ATOMIC_FLAG_INIT`, it is unspecified whether an `atomic_flag` object has an initial state of set or clear.

```
bool atomic_flag_test(volatile atomic_flag* object) noexcept;
bool atomic_flag_test(atomic_flag* object) noexcept;
bool atomic_flag_test_explicit(volatile atomic_flag* object, memory_order order) noexcept;
bool atomic_flag_test_explicit(atomic_flag* object, memory_order order) noexcept;
bool atomic_flag::test(memory_order order = memory_order_seq_cst) volatile;
bool atomic_flag::test(memory_order order = memory_order_seq_cst) noexcept;
```

❓ *Requires:* The order argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

❓ *Effects:* Memory is affected according to the value of order.

❓ *Returns:* Atomically returns the value pointed to by object or this.

```
bool atomic_flag_test_and_set(volatile atomic_flag* object) noexcept;
bool atomic_flag_test_and_set(atomic_flag* object) noexcept;
bool atomic_flag_test_and_set_explicit(volatile atomic_flag* object,
                                       memory_order order) noexcept;
bool atomic_flag_test_and_set_explicit(atomic_flag* object, memory_order order) noexcept;
bool atomic_flag::test_and_set(memory_order order = memory_order_seq_cst) volatile;
bool atomic_flag::test_and_set(memory_order order = memory_order_seq_cst) noexcept;
```

- 5 *Effects:* Atomically sets the value pointed to by object or by this to true. Memory is affected according to the value of order. These operations are atomic read-modify-write operations (4.7).
- 6 *Returns:* Atomically, the value of the object immediately before the effects.


```
void atomic_flag_clear(volatile atomic_flag* object) noexcept;
void atomic_flag_clear(atomic_flag* object) noexcept;
void atomic_flag_clear_explicit(volatile atomic_flag* object,
                                memory_order order) noexcept;
void atomic_flag_clear_explicit(atomic_flag* object, memory_order order) r
void atomic_flag::clear(memory_order order = memory_order_seq_cst) volatil
void atomic_flag::clear(memory_order order = memory_order_seq_cst) noexcep
```

Requires: The order argument shall not be memory_order_consume, memory_order_acquire, nor memory_order_acq_rel.

Effects: Atomically sets the value pointed to by object or by this to false. Memory is affected according to the value of order.

Add a new subclause after [**atomics.fences**]:

- 1 The functions in this subclause provide a mechanism to wait for the value of an atomic object to change, more efficiently than can be achieved with polling. Waiting functions in this facility may block until they are unblocked by notifying functions, according to each function's effects. [*Note:* Programs are not guaranteed to observe transient atomic values, an issue known as the A-B-A problem, resulting in continued blocking if a condition is only temporarily met. – *End note*]
- 2 The functions `atomic_wait` and `atomic_wait_explicit` are waiting functions. The functions `atomic_notify_one` and `atomic_notify_all` are notifying functions.

```
template<class T>
    void atomic_notify_one(const volatile atomic<T>* object);
template<class T>
    void atomic_notify_one(const atomic<T>* object);
void atomic_notify_one(const volatile atomic_flag* object);
void atomic_notify_one(const atomic_flag* object);
```

- 3 *Effects:* Unblocks up to execution of a waiting function that blocked after observing the result of an atomic operation X, if there exists another atomic operation Y, such that X precedes Y in the modification order of `*object`, and Y happens before this call.

```
template<class T>
    void atomic_notify_all(const volatile atomic<T>* object);
template<class T>
    void atomic_notify_all(const atomic<T>* object);
void atomic_notify_all(const volatile atomic_flag* object);
void atomic_notify_all(const atomic_flag* object);
```

- 4 *Effects:* Unblocks each execution of a waiting function that blocked after observing the result of an atomic operation X, if there exists another atomic operation Y, such that X precedes Y in the modification order of `*object`, and Y happens before this call.

```
template<class T>
    void atomic_wait(const volatile atomic<T>* object,
                     typename atomic<T>::value_type old);
```

```
template<class T>
    void atomic_wait(const atomic<T>* object,
                     typename atomic<T>::value_type old);
```

```
void atomic_wait(const volatile atomic_flag* object,
                 bool old);
```

```
void atomic_wait(const atomic_flag* object,
                 bool old);
```

5 *Effects:* Equivalent to `atomic_wait_explicit(object, old, memory_order_seq_cst)`;

```
template<class T>
    void atomic_wait_explicit(const volatile atomic<T>* object,
                              typename atomic<T>::value_type old,
                              memory_order order);
```

```
template<class T>
    void atomic_wait_explicit(const atomic<T>* object,
                              typename atomic<T>::value_type old,
                              memory_order order);
```

6 *Requires:* The order argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

7 *Effects:* Repeatedly performs the following steps, in order:

- (7.1) — Evaluates `object->load(order) != old` then, if the result is `true`, returns.
- (7.2) — Blocks until an implementation-defined condition has been met. [*Note:* Consequently, it may unblock for reasons other than a call to a notifying function. — *end note*]

```
void atomic_wait_explicit(const volatile atomic_flag* object,
                          bool old,
                          memory_order order);
```

```
void atomic_wait_explicit(const atomic_flag* object,
                          bool old,
                          memory_order order);
```

8 *Requires:* The order argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

9 *Effects:* Repeatedly performs the following steps, in order:

- (9.1) — Evaluates `object->test(order) != old` then, if the result is `true`, returns.
- (9.2) — Blocks until an implementation-defined condition has been met. [*Note:* Consequently, it may unblock for reasons other than a call to a notifying function. — *end note*]

Modify Table 140 "Thread support library summary" in **[thread.general]** as follows:

Table 140 — Thread support library summary

Subclause		Header(s)
33.2	Requirements	
33.3	Threads	<thread>
33.4	Mutual exclusion	<mutex> <shared_mutex>
33.5	Condition variables	<condition_variable>
33.◆	Semaphores	<semaphore>
33.◆	Latches and barriers	<latch> <barrier>
33.6	Futures	<future>

Add two new subclauses after **[thread.condition]**:

33.◆ Semaphores

[thread.semaphore]

- 1 Semaphores are lightweight synchronization primitives used to constrain concurrent access to a shared resource. They are widely used to implement other synchronization primitives and, whenever both are applicable, can be more efficient than condition variables.
- 2 A counting semaphore is a semaphore object that models a non-negative resource count. A binary semaphore is a semaphore object that has only two states, also known as available and unavailable. [*Note:* A binary semaphore should be more efficient than a counting semaphore with a unit magnitude count. — *end note*]

33.◆.1 Header <semaphore> synopsis

[thread.semaphore.syn]

```
namespace std {
    template<ptrdiff_t least_max_value = implementation-defined>
        class counting_semaphore;

    using binary_semaphore = counting_semaphore<1>;
}
```

33.2 Class template `counting_semaphore`

[`thread.semaphore.counting.class`]

```
namespace std {
    template<ptrdiff_t least_max_value>
    class counting_semaphore {
    public:
        static constexpr ptrdiff_t max() noexcept;

        explicit constexpr counting_semaphore(ptrdiff_t);
        ~counting_semaphore();

        counting_semaphore(const basic_semaphore&) = delete;
        counting_semaphore(basic_semaphore&&) = delete;
        counting_semaphore& operator=(const basic_semaphore&) = delete;
        counting_semaphore& operator=(basic_semaphore&&) = delete;

        void release(ptrdiff_t update = 1);
        void acquire();
        bool try_acquire();
        template<class Clock, class Duration>
            bool try_acquire_until(const chrono::time_point<Clock, Duration>&);
        template<class Rep, class Period>
            bool try_acquire_for(const chrono::duration<Rep, Period>&);

    private:
        ptrdiff_t counter; // exposition only
    };
}
```

- 1 Class `counting_semaphore` maintains an internal counter that is initialized when the semaphore is created. Threads may block waiting until `counter >= 1`.
- 2 Semaphores permit concurrent invocation of the `release`, `acquire`, `try_acquire`, `try_acquire_for`, and `try_acquire_until` member functions.

`static constexpr ptrdiff_t max() noexcept;`

- 3 *Returns:* The maximum value of `counter`. This value shall not be less than that of the template argument `least_max_value`. [*Note:* The value may exceed `least_max_value`. – *end note*]

`explicit constexpr counting_semaphore(ptrdiff_t desired);`

- 4 *Requires:* `desired >= 0` and `desired <= max()`.

5 *Effects:* counter = desired.

~counting_semaphore();

6 *Requires:* For every function call that blocks on counter, a function call that will cause it to unblock and return shall happen before this call. [*Note:* This relaxes the usual rules, which would have required all wait calls to happen before destruction. — *end note*]

7 *Effects:* Destroys the object.

void release(ptrdiff_t update = 1);

8 *Requires:* update >= 0, and counter + update <= max().

9 *Effects:* counter += update, executed atomically. If any threads are blocked on counter, unblocks them.

10 *Synchronization:* Strongly happens before invocations of try_acquire that observe the result of the effects.

bool try_acquire();

11 *Effects:*

(11.1) — With low probability, returns immediately. [*Note:* An implementation should ensure that try_acquire does not consistently return false in the absence of contending acquisitions. — *end note*]

(11.2) — Otherwise, if counter >= 1, then counter -= 1 is executed atomically.

12 *Returns:* true if counter was decremented, otherwise false.

void acquire();

13 *Effects:* Repeatedly performs the following steps, in order:

(13.1) — Evaluates try_acquire, then, if the result is true, returns.

(13.2) — Blocks until counter >= 1.

template<class Clock, class Duration>

bool try_acquire_until(const chrono::time_point<Clock, Duration>& abs_time);

template<class Rep, class Period>

bool try_acquire_for(const chrono::duration<Rep, Period>& rel_time);

14 *Effects:* Repeatedly performs the following steps, in order:

(14.1) — Evaluates `try_acquire`. If the result is `true`, returns `true`.

(14.2) — Blocks until the timeout expires or `counter >= 1`. If the timeout expired, returns `false`.

15 *Throws*: Timeout-related exceptions (33.2.4).

33.◆ Coordination Types

[thread.coord]

- 1 This section describes various concepts related to thread coordination, and defines the **coordination types** `latch` and `barrier`. These types facilitate concurrent computation performed by a number of threads, in one or more phases.
- 2 In this subclause, a synchronization point represents a condition that a thread may contribute to or wait for, potentially blocking until it is satisfied. A thread arrives at the synchronization point when it has an effect on the state of the condition, even if it does not cause it to become satisfied.
- 3 Concurrent invocations of the member functions of coordination types, other than their destructors, do not introduce data races.

33.◆.1 Latches

[thread.coord.latch]

- 1 A latch is a thread coordination mechanism that allows any number of threads to block until an expected count is summed (exactly) by threads that arrived at the latch. The expected count is set when the latch is constructed. An individual latch is a single-use object; once the count has been reached, the latch cannot be reused.

33.◆.1.1 Header `<latch>` synopsis

[thread.coord.latch.syn]

```
namespace std {  
    class latch  
}
```

33.1.2 Class `latch`

[[thread.coord.latch.class](#)]

```
namespace std {
    class latch {
    public:
        explicit constexpr latch(ptrdiff_t expected);
        ~latch();

        latch(const latch&) = delete;
        latch(latch&&) = delete;
        latch& operator=(const latch&) = delete;
        latch& operator=(latch&&) = delete;

        void arrive(ptrdiff_t update = 1);
        bool try_wait() const noexcept;
        void wait() const;
        void arrive_and_wait(ptrdiff_t n update = 1);

    private:
        ptrdiff_t counter; // exposition only
    };
}
```

- 1 A `latch` maintains an internal counter that is initialized when the `latch` is created. Threads may block at the `latch`'s synchronization point, waiting for counter to be decremented to 0.

```
explicit constexpr latch(ptrdiff_t expected);
```

- 2 *Requires:* `expected >= 0`.

- 3 *Effects:* `counter = expected`.

```
~latch();
```

- 4 *Requires:* No threads are blocked at the synchronization point.

- 5 *Effects:* Destroys the `latch`.

- 6 *Remarks:* May be called even if some threads have not yet returned from functions that block at the synchronization point, provided that they are unblocked. [*Note:* The destructor may block until all threads have exited invocations of `wait` on this object. — *end note*]

```
void arrive(ptrdiff_t update);
```


7 *Requires:* `counter >= update` and `update >= 0`.

8 *Effects:* Atomically decrements `counter` by `update`.

9 *Synchronization:* Synchronizes with the returns from all calls unblocked by the effects.

10 *Remarks:* Arrives at the synchronization point with `update` count.

```
bool try_wait() const noexcept;
```

11 *Returns:* `counter == 0`.

```
void wait() const;
```

12 *Effects:* If `counter == 0`, returns immediately. Otherwise, blocks the calling thread at the synchronization point until `counter == 0`.

```
void arrive_and_wait(ptrdiff_t update);
```

13 *Effects:* Equivalent to `arrive(update); wait();`.

33.◆.2 Barriers

[[thread.coord.barrier](#)]

1 A barrier is a thread coordination mechanism that allows at most an expected count of threads to block until that count is summed (exactly) by threads that arrived at the barrier in each of its successive phases. Once threads are released from blocking at the synchronization point for a phase, they can reuse the same barrier immediately in its next phase. [*Note:* It is thus useful for managing repeated tasks, or phases of a larger task, that are handled by multiple threads. — *end note*]

2 A barrier has a completion step that is a (possibly empty) set of effects associated with a phase of the barrier. When the member functions defined in this subclause arrive at the barrier, they have the following effects:

(2.1) — When the expected number of threads for this phase have arrived at the barrier, one of those threads executes the barrier type's completion step.

(2.2) — When the completion step is completed, all threads blocked at the synchronization point for this phase are unblocked and the barrier enters its next phase. The end of the completion step strongly happens before the returns from all calls unblocked by its completion.

33.2.1 Header <barrier> synopsis

[thread.coord.barrier.syn]

```
namespace std {  
    template<class CompletionFunction = implementation-defined>  
        class barrier;  
}
```

33.2.2 Class template **barrier**

[thread.coord.barrier.class]

```
namespace std {
    template<class CompletionFunction>
    class barrier {
    public:
        using arrival_token = implementation-defined;

        explicit barrier(ptrdiff_t expected,
                        CompletionFunction f = CompletionFunction());
        ~barrier();

        barrier(const barrier&) = delete;
        barrier(barrier&) = delete;
        barrier& operator=(const barrier&) = delete;
        barrier& operator=(barrier&) = delete;

        [[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);
        bool try_wait(arrival_token) const;
        void wait(arrival_token) const;

        void arrive_and_wait();
        void arrive_and_drop();

    private:
        CompletionFunction completion; // exposition only
    };
}
```

- 1 A **barrier** is a barrier type with a completion step controlled by a function object. The completion step calls **completion**. Threads may block at the barrier's synchronization point for a phase, waiting for the expected sum contributions by threads that arrive in that phase.
- 2 **CompletionFunction** shall be CopyConstructible and **is_invocable_r_v<void, CompletionFunction>** shall be true.
- 3 **barrier::arrival_token** is an implementation-defined type that is MoveConstructible and MoveAssignable.

```
explicit barrier(ptrdiff_t expected, CompletionFunction f);
```

- 4 *Requires:* **expected** ≥ 0 , and **noexcept(f())** shall be true.

- 5 *Effects:* Initializes the barrier for expected number of threads in the first phase, and initializes completion with `std::move(f)`. [*Note:* If `expected` is 0 this object may only be destroyed. — *end note*]

`~barrier();`

- 6 *Requires:* No threads are blocked at a synchronization point for any phase.

- 7 *Effects:* Destroys the barrier.

- 8 *Remarks:* May be called even if some threads have not yet returned from functions that block at a synchronization point, provided that they have unblocked. [*Note:* The destructor may block until all threads have exited invocations of `wait()` on this object. — *end note*]

`[[nodiscard]] arrival_token arrive(ptrdiff_t update);`

- 9 *Requires:* The expected count is not less than `update`.

- 10 *Effects:* Constructs an object of type `arrival_token` that is associated with the `barrier`'s synchronization point for the current phase, then arrives `update` times at the synchronization point for the current phase.

- 11 *Synchronization:* The call to `arrive` strongly happens before the start of the completion step for the current phase.

- 12 *Returns:* The constructed object.

- 13 *Remarks:* This may cause the completion step to start.

`bool try_wait(arrival_token arrival) const;`

- 14 *Requires:* `arrival` is associated with a synchronization point for the current or the immediately preceding phases of the `barrier`.

- 15 *Returns:* `true` if the synchronization point condition associated with `arrival` is satisfied, otherwise `false`.

`void wait(arrival_token arrival) const;`

- 16 *Requires:* `arrival` is associated with a synchronization point for the current or the immediately preceding phases of the `barrier`.

- 17 *Effects:* Blocks at the synchronization point associated with `arrival` until the condition is satisfied.

```
void arrive_and_wait();
```

18 *Effects:* Equivalent to `wait(arrive())`.

```
void arrive_and_drop();
```

19 *Requires:* The expected number of threads for the current phase is not 0.

20 *Effects:* Decrements the expected number of threads for subsequent phases by 1, then arrives at the synchronization point for the current phase.

21 *Synchronization:* The call to `arrive_and_drop` strongly happens before the start of the completion step for the current phase.

22 *Remarks:* This may cause the completion phase to start.

Create the following feature test macros:

- `__cpp_lib_atomic_lock_free_type_aliases`, which implies that `atomic_signed_lock_free` and `atomic_unsigned_lock_free` types are available.
- `__cpp_lib_atomic_flag_test`, which implies the test methods and free functions for `atomic_flag` are available.
- `__cpp_lib_atomic_wait`, which implies the `notify_*` and `wait` methods and free functions for `atomic` and `atomic_flag` and the `atomic_int_fast_wait_t` and `atomic_uint_fast_wait_t` types are available.
- `__cpp_lib_semaphore`, which implies that `counting_semaphore` and `binary_semaphore` are available.
- `__cpp_lib_latch`, which implies that `latch` is available.
- `__cpp_lib_barrier`, which implies that `barrier` is available.

§ Index

§ Terms defined by this specification

[coordination types](#), in §2